

Faculty of Engineering and Technology
Object Oriented Programming (CEUE203)
A.Y 2026 – 27 Semester – 3
Practical List

Practical Number		CO/PO
6	Part A — Practice Programs (Hour 1) Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. 1. Remote control: an interface Switchable with on()/off() and a default toggle(); Fan and Light implement it. Loop over a Switchable[] toggling each. Then add a functional interface that decides, given a device and an hour, whether it may switch on, and supply it once as an anonymous class and once as a lambda. 2. Notification senders: a functional interface Notifier with send(String); make email and SMS senders as lambdas, hold them in an array, and broadcast one message through all. Add a marker interface Urgent and send twice for senders marked urgent. 3. (For Advanced Learners – home) Build a small discount engine where each rule is a lambda the user picks at runtime and applies to a list of prices; organise it into packages and ship it as a runnable JAR. Part B — Project Work (Hour 2): MiniBank With several classes now in place, this lab gives MiniBank a clean structure. You express account capabilities as interfaces, organise the code into model, util and service packages, and build a runnable JAR — the form in which the finished application will actually be delivered and run. 1. Create an interface named Transactable with methods void deposit(long amount) and boolean withdraw(long amount); make Account implement it. 2. Create an interface named InterestBearing with a default method double yearlyInterest() that uses interestRate() and the balance. 3. Create a functional interface named WithdrawRule with a single method boolean allow(Account account, long amount). Use it two ways in main: first with an anonymous class, then with a lambda expression. 4. Create a marker interface named Premium (an interface with no methods). 5. Move all classes into the packages model, util and service (add package statements and fix the import statements). Use one static import. 6. Build a runnable JAR file with a manifest whose Main-Class is the MiniBank class, and run it with the command java -jar. Key Questions / Analysis / Interpretation to be evaluated during/after Implementation 1. What is the difference between an interface and an abstract class? 2. What is a functional interface, and how does a lambda use it? 3. Why is a default method useful in an interface? Supplementary Problems – • Add a second default method to InterestBearing and call it from main. • Add a static import for the Validator methods and use them without the class name. Key Skills to be addressed – Interfaces; default, static and private interface methods; functional interfaces with lambdas; anonymous classes; marker interfaces; packages; static import; building a JAR. Applications – Interface-based design and packaging are the basis of every maintainable, modular codebase and of plug-in architectures. Learning Outcome – The student can design interface-based code, organise it into packages, and produce a runnable build artifact. (CO2) Dataset / Test Data (Source and Description, if applicable): No external dataset. Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git. Total Hours of Problem Definition Implementation: 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work.	CO2 PO1, PO3, PSO2

Practical Number		CO/PO
	• Total Hours of Engagement = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	
	Post Laboratory Work Description: Commit and tag lab-06. Confirm the project still compiles after moving to packages and that java -jar runs the program.	
	Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Correct interfaces (incl. functional and marker) – 35%; package organisation and static import – 25%; runnable JAR – 20%; naming conventions and viva – 20%.	
	Expected Input / Process / Output Input: In code: a WithdrawRule implemented first as an anonymous class, then as a lambda; the built JAR run from the command line. Process: The anonymous class and the lambda both provide the allow() logic; the JAR manifest tells java which class has main. Expected Output: Both versions return the same allow() result for the same inputs; java -jar minibank.jar launches the menu from Practical 1.	